

DD2440 Advanced Algorithms

Fall 2026

Lecture 1: Course setup and Intro to Approx
Algorithms

Ioana-Oriana Bercea

2026-09-8

Agenda

Vertex Cover

Course setup

Approximation Algorithms

Vertex Cover

Agenda

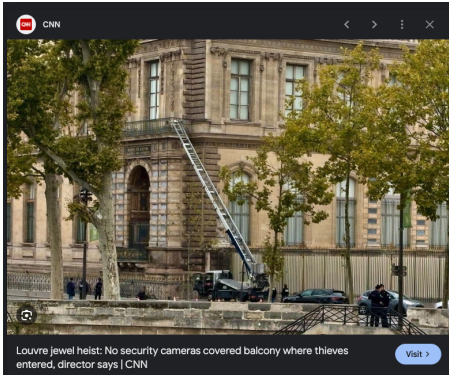
Vertex Cover

Course setup

Approximation Algorithms

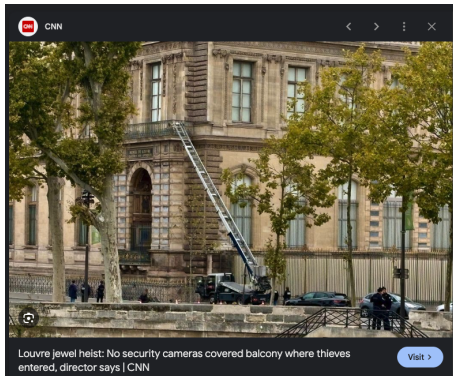
Vertex Cover

A Cautionary Tale: The Louvre Heist



"No security cameras covered the balcony where thieves entered..."

A Cautionary Tale: The Louvre Heist



- ▶ **The Failure:** Critical entry points were left completely unmonitored.
- ▶ **The Cause:** Poor camera placement and coverage planning.
- ▶ **The Question:** How do we guarantee *every* corridor and entry point is watched using the *fewest* cameras possible?

Modeling Security Placement as a Graph

We can abstract the museum security network into a graph

$G = (V, E)$:

- ▶ **Vertices (V):** Potential camera locations (room intersections, doorways, balconies).
- ▶ **Edges (E):** Corridors, hallways, or entry paths that thieves could use.
- ▶ **Coverage Rule:** Placing a camera at vertex $v \in V$ monitors all edges connected to v .

The Security Dilemma

- ▶ **Requirement:** Every edge $e \in E$ must be watched by at least one camera.
- ▶ **Constraint:** Budget limits force us to minimize the total number of cameras.

Vertex Cover

The requirement: A vertex cover in a graph G is a set of vertices that *covers* (“touches”) every edge of G .

The optimization: Output a vertex cover of **minimum** size.

More formally:

Input: Graph $G = (V, E)$

Output: A vertex cover $S \subseteq V$ of G

Goal: Minimize size of S

Vertex Cover

The requirement: A vertex cover in a graph G is a set of vertices that *covers* (“touches”) every edge of G .

The optimization: Output a vertex cover of **minimum** size.

More formally:

Input: Graph $G = (V, E)$

Output: A vertex cover $S \subseteq V$ of G

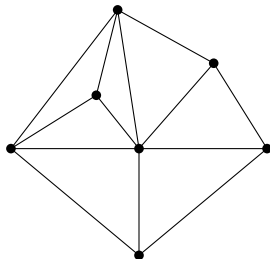
Goal: Minimize size of S

Poll: have you seen this problem before?

Vertex Cover

The requirement: A vertex cover in a graph G is a set of vertices that *covers* (“touches”) every edge of G .

The optimization: Output a vertex cover of **minimum** size.



How many vertices do we need to choose? (Talk to your neighbor and say a number! (1 min or so))

About this course

Focus: design and analyze modern algorithms

- ▶ design
 - ▶ we will learn a set of general *techniques* for algorithm design
 - ▶ we will see examples of how these techniques can be applied to various problems
 - ▶ **Goal:** apply these techniques to *new* problems (conceptual understanding, also in assignments)

About this course

Focus: design and analyze modern algorithms

- ▶ design

- ▶ we will learn a set of general *techniques* for algorithm design
- ▶ we will see examples of how these techniques can be applied to various problems
- ▶ **Goal:** apply these techniques to *new* problems (conceptual understanding, also in assignments)

- ▶ analyze

- ▶ we will *mathematically prove* claims about the performance of our algorithms: how do they scale? are they reliable?
- ▶ claims about correctness or efficiency, e.g., running time, quality of solutions output, space that they require
- ▶ **Goal:** be able to select *the best* among many algorithms and *convincingly argue* why they are the best

Modern algorithms

What do we mean by *modern* algorithms?

- ▶ modern *principles and models of computation*
 - ▶ computers don't look the same any more, e.g., huge hard disks (Big Data), distributed (data center, cloud computing)
 - ▶ polynomial time is no longer enough, e.g., sublinear is much better
 - ▶ *randomization* will feature quite heavily

Modern algorithms

What do we mean by *modern* algorithms?

- ▶ modern *principles and models of computation*
 - ▶ computers don't look the same any more, e.g., huge hard disks (Big Data), distributed (data center, cloud computing)
 - ▶ polynomial time is no longer enough, e.g., sublinear is much better
 - ▶ *randomization* will feature quite heavily
- ▶ modern *problems*
 - ▶ we will study mainly *optimization* problems
 - ▶ a lot of interesting problems are intractable (NP-hard or worse), so *approximation* will be key
 - ▶ understand some of the modern, *realistic* challenges in algorithm design

Overarching goals

My personal goals are to show you:

- ▶ techniques that you can use later on when you go out into the "real world"
 - ▶ they are general and behind a lot of modern algorithms
 - ▶ some of these questions may pop up as interview questions
- ▶ work that is also interesting to the industry (e.g., Google Research, Facebook research, Apache etc.)
- ▶ we have some room to decide what we want to look at

Tentative topics

Part 1: Deterministic Algorithms (Lectures 1-4)

- ▶ Approximation Algorithms (greedy, submodular optimization for influence maximization)
- ▶ Fixed-Parameter Tractable (FPT) Algorithms
- ▶ Fast Fourier Transform

Part 2: Randomized Algorithms (Lectures 5-10)

- ▶ Sampling
- ▶ Randomized Exact, Approximation, FPT Algorithms
- ▶ Hashing and dimensionality reduction

Part 3: Miscellaneous (Lectures 11-15)

- ▶ Streaming algorithms (e.g., extreme small space algorithms)
- ▶ Multiplicative weights algorithms (e.g., prediction with expert advice)
- ▶ Algorithms for specific real-world graphs (e.g., social networks)

Tentative topics

Part 1: Deterministic Algorithms (Lectures 1-4)

- ▶ Approximation Algorithms (greedy, submodular optimization for influence maximization)
- ▶ Fixed-Parameter Tractable (FPT) Algorithms
- ▶ Fast Fourier Transform

Part 2: Randomized Algorithms (Lectures 5-10)

- ▶ Sampling
- ▶ Randomized Exact, Approximation, FPT Algorithms
- ▶ Hashing and dimensionality reduction

Part 3: Miscellaneous (Lectures 11-15)

- ▶ Streaming algorithms (e.g., extreme small space algorithms)
- ▶ Multiplicative weights algorithms (e.g., prediction with expert advice)
- ▶ Algorithms for specific real-world graphs (e.g., social networks)
- ▶ What CS problems do you care about? What would you be interested in learning?

What CS problems do you care about? What would you be interested in learning?

<https://www.menti.com/alrixnu9fooa>

Code: 8569 5803



Lectures

- ▶ lectures start at quarter past and run in two chunks of 45 minutes each, with a 15 min break in between
- ▶ *during the lecture*: slides + extra explanations on the board (all required material will be included in the slides)
- ▶ *after the lecture*: slides + extra reading material (for alternate explanations - no main textbook, so other lectures notes or surveys)
- ▶ please reach out to us at avalg-help@kth.se for the fastest reply and come to office hours

Course Assessment Strategy & Guidelines

Assessment	Format	GenAI?	Key Weight / Prerequisite
Individual Assignments	3 sets + Interview + Peer Review	No	Must pass 2 problems across diff. sets
Group Assignments	3 sets (Groups up to 4)	Yes	$P/F/F_x$ practice for individual sets
Projects	Kattis + Report + Interview	—	E–A grades (e.g., TSP 2D & MSF)

Important Policy Note

Passing the course requires completing all group assignments, passing two individual assignment problems from different sets, and passing one project.

Individual assignments: 3 sets + interview + peer reviews

- ▶ each assignment consists of 3 questions (one for A, one for C, and one for E) + peer reviews + interviews
- ▶ to be completed **individually**, cannot use GenAI
- ▶ **the interview**: at the end of the course, **in person**: questions about your submission to the individual assignments
- ▶ (before interview)
 - ▶ you will mainly get the grade you submit for or an F
 - ▶ rare partial credit (exceptional but incorrect solutions), e.g., if you submit for an *A*, you might get a *B* (but not a *C*)
 - ▶ you might get an Fx (require revision or you missed peer reviews)
- ▶ (after interview): grade from the first pass is finalized (or it's an F)
- ▶ NB! People who failed this course usually failed this part.

Exercises and Group assignments: 3 sets

Goal: practice for the individual assignments

Group assignments: 3 sets

- ▶ P/F/Fx, GenAi is allowed (see specific guidelines on Canvas)
- ▶ group of up to 4 people : **start making your groups and adding them to Canvas**
- ▶ hints are released halfway through the week

Exercises: 3 sets

- ▶ *before each session:* we will release some exercises
- ▶ you have 4 options (3 on campus, one Zoom)
- ▶ *during each session:* work together with the TA on the exercises + discuss solutions to past assignments

Projects

- ▶ in a group: search for algorithms and implement them for some problem
 - ▶ evaluation through Kattis + report + interview (E-A grades)
 - ▶ Traveling Salesman Problem 2D (i.e., approx algorithms): runtime and programming language matter (recommend: C/C++), required time varies
 - ▶ Minimum Spanning Forest (i.e., sampling): running time does not matter much
- ▶ honors: *independently* understand paper(s) really well
 - ▶ presentation + report
 - ▶ only for grade A, grading might be subjective
- ▶ **Goal:** you will develop skills to explore algorithms independently and get a better idea of what basics you need
- ▶ **It takes time (> 1 month), so start now!**

Final Grade Calculation Strategy

Target Grade	Prerequisites (To Pass)	Final Grade Formula
Pass (E)	Pass all 3 Group Assignments Pass 2 Individual Problems (diff. sets) Pass 1 Project	Mean of 3 assignments: • All 3 Individual Assignments OR
Higher (D–A)	Meet all Pass criteria above	• Best 2 Individual Assignments + max(Project, Honors Project)

Grading Rules to Remember

- ▶ **Individual Submissions:** Submit for an A $\rightarrow$ get A, B (rare partial credit), Fx, or F.
- ▶ **Project Grade:** Can boost your final average or substitute your lowest individual assignment set score.
- ▶ **Make-ups:** Available in January 2027.

Key deadlines

- ▶ Group Assignments: Monday (8:00) – Friday (19:00)
- ▶ Individual Assignments:
 - ▶ problems posted Friday at 08:00
 - ▶ submissions due **next** Friday at 13:00
 - ▶ peer review due the following Monday at 13:00
- ▶ Projects
 - ▶ two submission rounds
 - ▶ pick the ones that fits your schedule best
 - ▶ first round does not have peer reviews

Check Canvas for exact dates!

Get back to us ASAP if there are issues **especially with the in person interview**

Staff

Lectures: me (Ioana-Oriana Bercea) + Aris Gionis

- ▶ Office hours Tuesdays 13:00-14:00 (and by email)

Teaching Assistants (teach exercise classes and grade assignments/projects)

- ▶ some will also hold office hours, check schedule

Two student representatives: your chance to give feedback on the course as its going

- ▶ volunteers?
- ▶ we will meet two or three times in total

What are your hopes and concerns for the class?

<https://www.menti.com/a1819ca7esdj>

Code: 2398 1088



Feedback and help

- ▶ Canvas discussion board (non-private)
 - ▶ these get priority since they benefit everyone, anonymous feedback possible
- ▶ email avalg-help@kth.se (private, lecturer and TAs will see it)
 - ▶ fastest way to get an answer from one of us (including me)
- ▶ If you want to contest your grade...
 - ▶ send request to avalg-help@kth.se (we all discuss it)
 - ▶ each assignment will have a *specific deadline* until which you can contest your grade (you will know when)
 - ▶ this is so that we keep ourselves accountable to you and manage everything in time
- ▶ I endorse the CMU SCS Reasonable Person Principle
 - ▶ be reasonable and assume everyone else is trying to be reasonable as well <https://www.cs.cmu.edu/~weigand/staff/>

Advice from last year

- ▶ "Take the group assignments seriously. It will increase your chances of passing the individual assignments tenfold"
- ▶ "Start with the individual assignments the very same day they are released. Just reading it once or twice the same day is enough for you to start thinking in the right directions. Almost everything you need for the individual assignments is provided in the lecture slides (at least this time around)": *start early, keep thinking logically*
- ▶ "Refresh probability theory and graph theory before you start"
- ▶ "Go to the lectures, in the group assignments meet up and use a whiteboard explain concepts to each other, this will help you in the individual assignments. Start the individual assignments in time, it is possible to get a high grade in this course": *practice*
- ▶ "Work early, ask the teachers when you get stuck, and keep your spirit up for the whole course. It is far from easy, but still manageable with proper planning and work": *come to office hours and ask questions during class!*
- ▶ "Old homeworks and solutions are helpful": *check the assignments page*

What are your contributions to the class?

<https://www.menti.com/aliinw6y74bw>

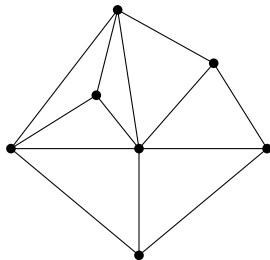
Code: 3790 9771



Vertex Cover

The requirement: A vertex cover in a graph G is a set of vertices that *covers* (“touches”) every edge of G .

The optimization: Output a vertex cover of **minimum** size.



How many vertices do we need to choose? (Talk to your neighbor and say a number! (1 min or so))

Agenda

Vertex Cover

Course setup

Approximation Algorithms

Vertex Cover

What is Optimization?

- ▶ an optimization problem is about making the **best possible/optimal** choice from a set of available choices
 - ▶ e.g.: what is the fastest way to visit cities A , B , C and D such that I return where I started? (TSP)
- ▶ The available alternatives are dictated by a set of **constraints**
 - ▶ e.g.: I only want to take streets. (TSP on planar graphs)
- ▶ In mathematical terms, we have an **objective function** and we want to either :
 - ▶ Minimize the function (e.g., cost, error, time): I want to minimize the distance I travel
 - ▶ Maximize the function (e.g., profit, efficiency, yield): I want to maximize the reward I get (each city has a reward)

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms
- ▶ Heuristics

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms
- ▶ Heuristics
- ▶ Average-case / smoothed analysis

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms
- ▶ Heuristics
- ▶ Average-case / smoothed analysis
- ▶ Approximation algorithms

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms
- ▶ Heuristics
- ▶ Average-case / smoothed analysis
- ▶ Approximation algorithms
- ▶ Parameterized algorithms

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms
- ▶ Heuristics
- ▶ Average-case / smoothed analysis
- ▶ Approximation algorithms
- ▶ Parameterized algorithms

← *today*

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms
- ▶ Heuristics
- ▶ Average-case / smoothed analysis
- ▶ Approximation algorithms
- ▶ Parameterized algorithms

← *today*

← *later lecture*

Dealing with intractability

A lot of problems we care about are NP-hard (or worse). How to deal with these?

- ▶ Exact exponential algorithms
- ▶ Heuristics
- ▶ Average-case / smoothed analysis
- ▶ Approximation algorithms
- ▶ Parameterized algorithms

← *TSP project*

← *today*

← *later lecture*

Approximation Algorithms

Definition 1.

An algorithm for an optimization problem has **approximation ratio** α if it always finds a solution with value within a factor α of the optimum value.

Approximation Algorithms

Definition 1.

An algorithm for an optimization problem has **approximation ratio** α if it always finds a solution with value within a factor α of the optimum value.

Notation:

- ▶ **Alg**(I) = value of solution found by algorithm on input I
- ▶ **Opt**(I) = value of optimal solution for input I

Approximation Algorithms

Definition 1.

An algorithm for an optimization problem has **approximation ratio** α if it always finds a solution with value within a factor α of the optimum value.

- ▶ For **maximization** problems: $\mathbf{Alg}(I) \geq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \leq 1$)

Notation:

- ▶ $\mathbf{Alg}(I)$ = value of solution found by algorithm on input I
- ▶ $\mathbf{Opt}(I)$ = value of optimal solution for input I

Approximation Algorithms

Definition 1.

An algorithm for an optimization problem has **approximation ratio** α if it always finds a solution with value within a factor α of the optimum value.

- ▶ For **maximization** problems: $\mathbf{Alg}(I) \geq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \leq 1$)
- ▶ For **minimization** problems: $\mathbf{Alg}(I) \leq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \geq 1$)

Notation:

- ▶ $\mathbf{Alg}(I)$ = value of solution found by algorithm on input I
- ▶ $\mathbf{Opt}(I)$ = value of optimal solution for input I

Approximation Algorithms

Definition 1.

An algorithm for an optimization problem has **approximation ratio** α if it always finds a solution with value within a factor α of the optimum value.

- ▶ For **maximization** problems: $\mathbf{Alg}(I) \geq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \leq 1$)
- ▶ For **minimization** problems: $\mathbf{Alg}(I) \leq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \geq 1$)
- ▶ ...for every possible instance I of the problem.

Notation:

- ▶ $\mathbf{Alg}(I)$ = value of solution found by algorithm on input I
- ▶ $\mathbf{Opt}(I)$ = value of optimal solution for input I

Approximation Algorithms

Definition 1.

An algorithm for an optimization problem has **approximation ratio** α if it always finds a solution with value within a factor α of the optimum value.

- ▶ For **maximization** problems: $\mathbf{Alg}(I) \geq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \leq 1$)
- ▶ For **minimization** problems: $\mathbf{Alg}(I) \leq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \geq 1$)
- ▶ ...for every possible instance I of the problem.

Notation:

- ▶ $\mathbf{Alg}(I)$ = value of solution found by algorithm on input I
- ▶ $\mathbf{Opt}(I)$ = value of optimal solution for input I
- ▶ Almost always drop I and just write $\mathbf{Alg}$ and $\mathbf{Opt}$

Approximation Algorithms

Definition 1.

An algorithm for an optimization problem has **approximation ratio** α if it always finds a solution with value within a factor α of the optimum value.

- ▶ For **maximization** problems: $\mathbf{Alg}(I) \geq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \leq 1$)
- ▶ For **minimization** problems: $\mathbf{Alg}(I) \leq \alpha \cdot \mathbf{Opt}(I)$ ($\alpha \geq 1$)
- ▶ ...for every possible instance I of the problem.

Notation:

- ▶ $\mathbf{Alg}(I)$ = value of solution found by algorithm on input I
- ▶ $\mathbf{Opt}(I)$ = value of optimal solution for input I
- ▶ Almost always drop I and just write **Alg** and **Opt**

(Note: another common convention is to switch for maximization problems to always make $\alpha \geq 1$. See [Wikipedia article](#).)

Agenda

Vertex Cover

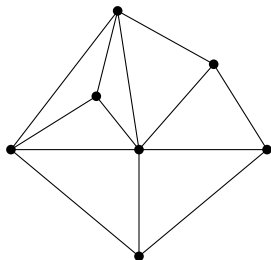
Course setup

Approximation Algorithms

Vertex Cover

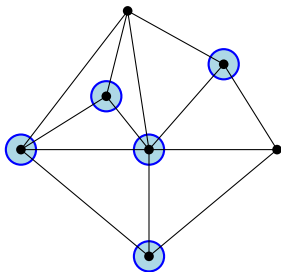
Vertex Cover

A vertex cover in a graph G is a set of vertices that *covers* (“touches”) every edge of G .



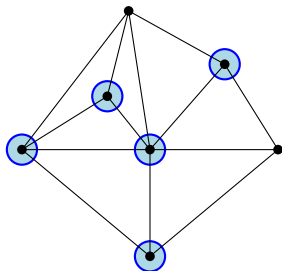
Vertex Cover

A vertex cover in a graph G is a set of vertices that *covers* (“touches”) every edge of G .



Vertex Cover

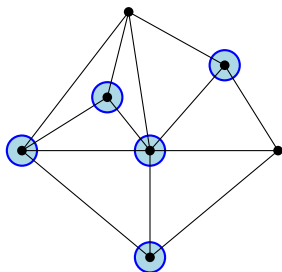
A vertex cover in a graph G is a set of vertices that *covers* (“touches”) every edge of G .



How many vertices do we need to choose?

Vertex Cover

A vertex cover in a graph G is a set of vertices that *covers* (“touches”) every edge of G .



How many vertices do we need to choose?

What is size of smallest vertex cover?

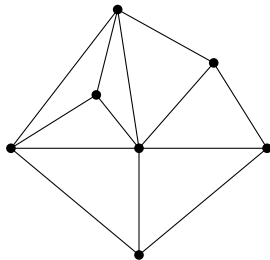
Vertex Cover

More formally:

Input: Graph $G = (V, E)$

Output: A vertex cover $S \subseteq V$ of G

Goal: Minimize size of S



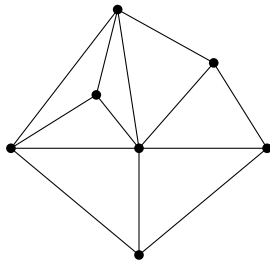
Vertex Cover

More formally:

Input: Graph $G = (V, E)$

Output: A vertex cover $S \subseteq V$ of G

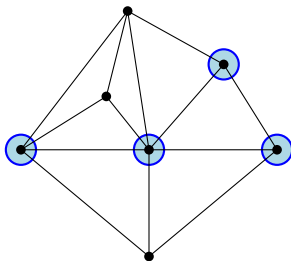
Goal: Minimize size of S



Poll: have you seen this problem before?

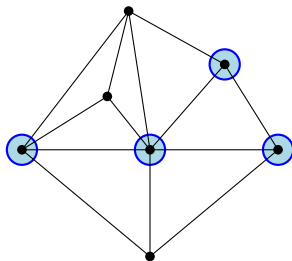
Test yourself: Vertex Cover

Is this a vertex cover of G ?



Test yourself: Vertex Cover

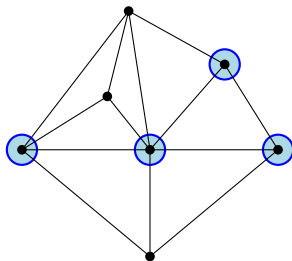
Is this a vertex cover of G ?



What is minimum vertex cover in this graph?

Test yourself: Vertex Cover

Is this a vertex cover of G ?



What is minimum vertex cover in this graph?

How did you do it?

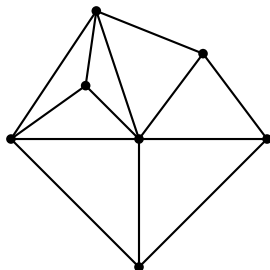
Approximation Algorithm for Minimum Vertex Cover

```
1 function MinVC( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4      $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

Approximation Algorithm for Minimum Vertex Cover

```
1 function MinVC( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4      $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

Example run of algorithm:



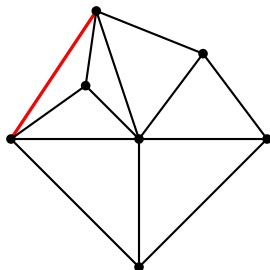
Approximation Algorithm for Minimum Vertex Cover

```

1 function MinVC( $G$ ):
2   Let  $S = \emptyset$ 
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$ 
4      $S \leftarrow S \cup \{u, v\}$ 
5   return  $S$ 

```

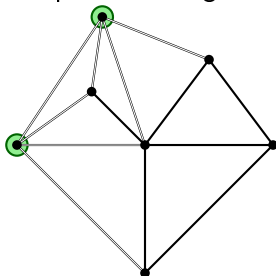
Example run of algorithm:



Approximation Algorithm for Minimum Vertex Cover

```
1 function MinVC( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4      $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

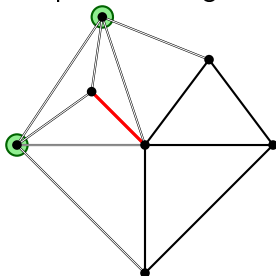
Example run of algorithm:



Approximation Algorithm for Minimum Vertex Cover

```
1 function MinVC( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4      $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

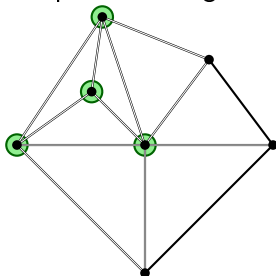
Example run of algorithm:



Approximation Algorithm for Minimum Vertex Cover

```
1 function MinVC( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4      $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

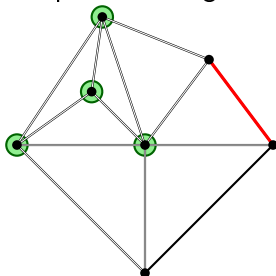
Example run of algorithm:



Approximation Algorithm for Minimum Vertex Cover

```
1 function MinVC( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4      $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

Example run of algorithm:



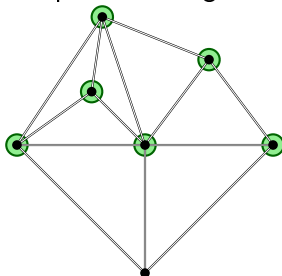
Approximation Algorithm for Minimum Vertex Cover

```

1 function MinVC( $G$ ):
2   Let  $S = \emptyset$ 
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$ 
4      $S \leftarrow S \cup \{u, v\}$ 
5   return  $S$ 

```

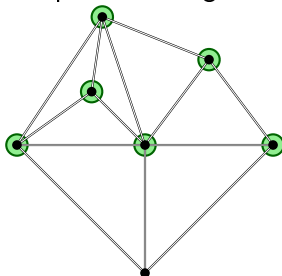
Example run of algorithm:



Approximation Algorithm for Minimum Vertex Cover

```
1 function MinVC( $G$ ):  
2   Let  $S = \emptyset$   
3   while exists an edge  $e = (u, v)$  such that  $u \notin S$  and  $v \notin S$   
4      $S \leftarrow S \cup \{u, v\}$   
5   return  $S$ 
```

Example run of algorithm:



Picked 6 vertices (whereas optimal solution only needs 4).

Analyzing the Algorithm

- ▶ Consider an optimal vertex cover S^* .
- ▶ For each edge $e = (u, v)$, at least one of u or v must be present in S^*
- ▶ This means that when we add u and v to our set S in the algorithm, we are adding at least one element of S^* , and at most one element that is not in S^* .
- ▶ So in our final set, at least half of the elements of S are also present in S^* (i.e., $|S \cap S^*| \geq |S|/2$).
- ▶ It follows that $\mathbf{Alg} = |S| \leq 2|S^*| = 2\mathbf{Opt}$

So the algorithm is a 2-approximation algorithm!

Can We Do Better?

At first glance the algorithm may feel silly:

- Why pick both ends of edges when we only need to pick one?
- Can't we improve by just picking one of the endpoints?

Can We Do Better?

At first glance the algorithm may feel silly:

- Why pick both ends of edges when we only need to pick one?
- Can't we improve by just picking one of the endpoints?

Natural greedy algorithm:

Pick the vertex which covers the largest number of uncovered edges

Can We Do Better?

At first glance the algorithm may feel silly:

- Why pick both ends of edges when we only need to pick one?
- Can't we improve by just picking one of the endpoints?

Natural greedy algorithm:

Pick the vertex which covers the largest number of uncovered edges

What is the approximation ratio of this algorithm?

Can We Do Better?

At first glance the algorithm may feel silly:

- Why pick both ends of edges when we only need to pick one?
- Can't we improve by just picking one of the endpoints?

Natural greedy algorithm:

Pick the vertex which covers the largest number of uncovered edges

What is the approximation ratio of this algorithm?

Surprisingly(?) this seemingly more clever algorithm is actually *much worse* in the worst case: it has an approximation ratio of $\Theta(\log n)$

Aside: theory vs practice

What does it mean that the greedy algorithm has a worse approximation ratio than the naive one?

- ▶ The *worst case* for the naive algorithm is better than the *worst case* for the greedy.

Aside: theory vs practice

What does it mean that the greedy algorithm has a worse approximation ratio than the naive one?

- ▶ The *worst case* for the naive algorithm is better than the *worst case* for the greedy.

But this says nothing about how common the worst case is.

Aside: theory vs practice

What does it mean that the greedy algorithm has a worse approximation ratio than the naive one?

- ▶ The *worst case* for the naive algorithm is better than the *worst case* for the greedy.

But this says nothing about how common the worst case is.

- ▶ Maybe naive algo almost always gives a solution twice as bad as the optimum.

Aside: theory vs practice

What does it mean that the greedy algorithm has a worse approximation ratio than the naive one?

- ▶ The *worst case* for the naive algorithm is better than the *worst case* for the greedy.

But this says nothing about how common the worst case is.

- ▶ Maybe naive algo almost always gives a solution twice as bad as the optimum.
- ▶ Maybe greedy algo almost always gives an optimal solution.

Aside: theory vs practice

What does it mean that the greedy algorithm has a worse approximation ratio than the naive one?

- ▶ The *worst case* for the naive algorithm is better than the *worst case* for the greedy.

But this says nothing about how common the worst case is.

- ▶ Maybe naive algo almost always gives a solution twice as bad as the optimum.
- ▶ Maybe greedy algo almost always gives an optimal solution.

Which one should you use in practice?

Aside: theory vs practice (cont.)

Why not both?

Aside: theory vs practice (cont.)

Why not both?

Run both algorithms and take the best results:

- ▶ Use the naive one to get a baseline solution which is guaranteed to be OK (at most $2\times$ **Opt**)
- ▶ Use the greedy one as a heuristic to hopefully get a better solution

Aside: theory vs practice (cont.)

Why not both?

Run both algorithms and take the best results:

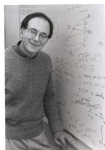
- ▶ Use the naive one to get a baseline solution which is guaranteed to be OK (at most $2 \times \mathbf{Opt}$)
- ▶ Use the greedy one as a heuristic to hopefully get a better solution

If you do the TSP project, you may want to use a strategy like this

- ▶ Use an approximation algorithm to get an initial solution.
- ▶ Use various heuristics to improve the initial solution or try to find other solutions.

Brief history of Approximating Vertex Cover

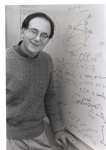
- ▶ [\[Karp'1972\]](#): 21 NP-hard problems



Richard Karp

Brief history of Approximating Vertex Cover

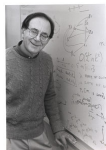
- ▶ [Karp'1972]: 21 NP-hard problems
- ▶ [Many'19xx]: $2 - o(1)$ algorithms



Richard Karp

Brief history of Approximating Vertex Cover

- ▶ [Karp'1972]: 21 NP-hard problems
- ▶ [Many'19xx]: $2 - o(1)$ algorithms
- ▶ [Håstad'2001]: no $7/6$ -approx assuming $\mathbf{P} \neq \mathbf{NP}$



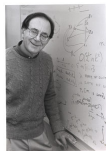
Richard Karp



Johan Håstad

Brief history of Approximating Vertex Cover

- ▶ [Karp'1972]: 21 NP-hard problems
- ▶ [Many'19xx]: $2 - o(1)$ algorithms
- ▶ [Håstad'2001]: no $7/6$ -approx assuming $\mathbf{P} \neq \mathbf{NP}$
- ▶ [Dinur-Safra'2005]: no 1.36-approx assuming $\mathbf{P} \neq \mathbf{NP}$



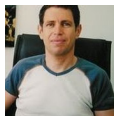
Richard Karp



Johan Håstad



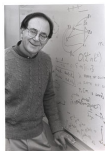
Irit Dinur



Muli Safra

Brief history of Approximating Vertex Cover

- ▶ [Karp'1972]: 21 NP-hard problems
- ▶ [Many'19xx]: $2 - o(1)$ algorithms
- ▶ [Håstad'2001]: no $7/6$ -approx assuming $\mathbf{P} \neq \mathbf{NP}$
- ▶ [Dinur-Safra'2005]: no 1.36-approx assuming $\mathbf{P} \neq \mathbf{NP}$
- ▶ [Khot-Regev'2008]: no $(2 - \epsilon)$ -approx assuming the “Unique Games Conjecture”



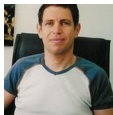
Richard Karp



Johan Håstad



Irit Dinur



Muli Safra



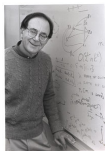
Subhash Khot



Oded Regev

Brief history of Approximating Vertex Cover

- ▶ [Karp'1972]: 21 NP-hard problems
- ▶ [Many'19xx]: $2 - o(1)$ algorithms
- ▶ [Håstad'2001]: no $7/6$ -approx assuming $\mathbf{P} \neq \mathbf{NP}$
- ▶ [Dinur-Safra'2005]: no 1.36-approx assuming $\mathbf{P} \neq \mathbf{NP}$
- ▶ [Khot-Regev'2008]: no $(2 - \epsilon)$ -approx assuming the “Unique Games Conjecture”
- ▶ [Khot-Minzer-Safra'2018]: no 1.41-approx assuming $\mathbf{P} \neq \mathbf{NP}$



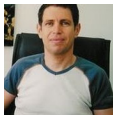
Richard Karp



Johan Håstad



Irit Dinur



Muli Safra



Subhash Khot



Oded Regev



Dor Minzer

Upcoming Week In The Course

- ▶ tomorrow 1pm-3pm: Lecture 2
Approximation algorithms (cont.)
- ▶ Get Prepared (no grade) assignment